tmux documentation

tmx SSH Dispatch — Operator Guide

Revision: 1 **Last modified:** 2026-05-22T14:30:00Z **Authority:** vasic-digital tmux project **Maintainer:** milosvasic **Scope:** Operator setup / verification / uninstall guide for `ssh <host>-tmx <session-name>` dispatch

---

1. Overview

`tmx-ssh-dispatch.sh` is the remote-side script that is wired into `~/.ssh/authorized_keys` via a `command="..."` directive. When you run

```
ssh nezha-tmx work
```

from your workstation, sshd on `nezha.local` authenticates your dispatch key, reads the `command=` directive, and exec's `tmx-ssh-dispatch.sh` with `SSH_ORIGINAL_COMMAND=work`. The dispatcher then:

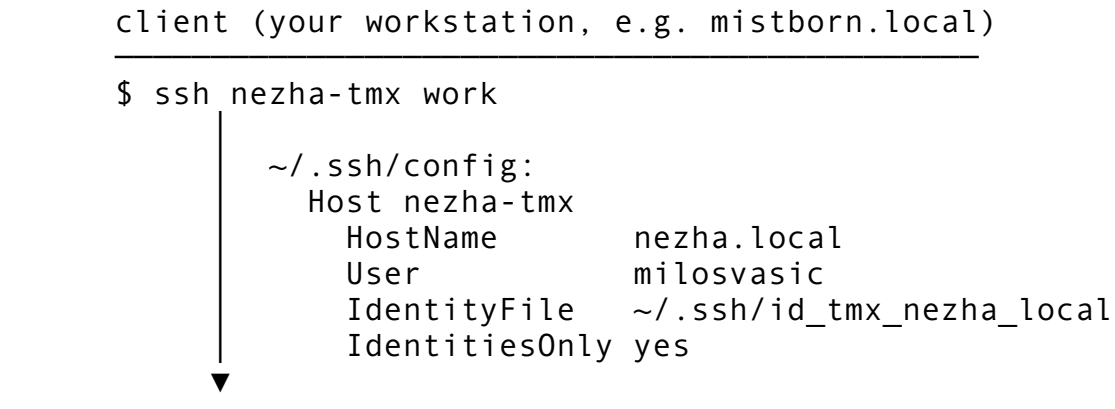
1. validates `work` against `^[A-Za-z0-9_.-]{1,64}$`;
2. queries `tmx-state recall work` for the session's last cwd;
3. tries `tmx attach -t work`, falling back to `tmx new -s work -c <last-pwd>`;
4. lands you directly inside that session.

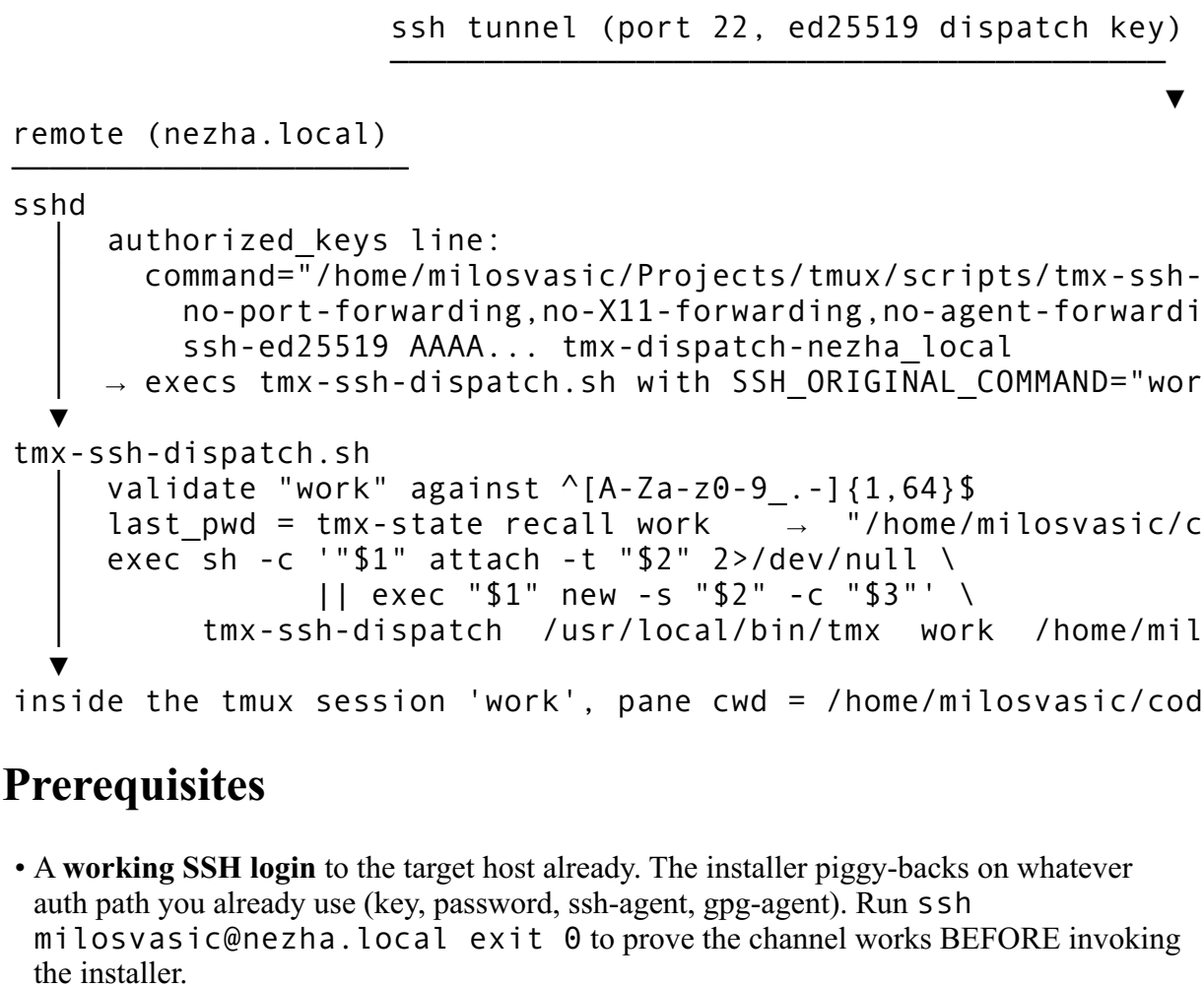
`ssh nezha-tmx` (no second argument) instead exec's `bash -l`, giving you a normal interactive login — same as a plain `ssh nezha`.

`tmx-ssh-install.sh` is the **client-side** bootstrapper that generates the dispatch key, copies the script to the remote, appends the `authorized_keys` entry, and writes a `Host nezha-tmx` alias to your local `~/.ssh/config`. It is idempotent: re-running it is a safe no-op.

Design authority: `docs/superpowers/specs/2026-05-22-tmx-shell-session-resume-design.md` §4.C + §4.D + §5.2.

2. Architecture





3. Prerequisites

- A **working SSH login** to the target host already. The installer piggy-backs on whatever auth path you already use (key, password, ssh-agent, gpg-agent). Run `ssh milosvasic@nezha.local exit 0` to prove the channel works BEFORE invoking the installer.
- The project cloned + built on **both** the client and the target host. The client only needs the `scripts/tmx-ssh-install.sh` and the `tmx-ssh-dispatch.sh.template`; the target needs the full repo with `bash scripts/setup.sh` GREEN.
- A modern `ssh-keygen` (any OpenSSH ≥ 8.0 — Apple ships this; on Linux any current distro).

4. Installation

4.1 One-line install (no remote project-path override)

```
bash scripts/tmx-ssh-install.sh milosvasic@nezha.local
```

What it does, step by step (announced on stderr):

```
[tmx-ssh-install] step 1: validate target (milosvasic@nezha.local) and rem
[tmx-ssh-install] step 2: ensure local key /Users/milosvasic/.ssh/id_tmx_n
[tmx-ssh-install] step 3: probe remote reachability via existing auth path
[tmx-ssh-install] reachable
[tmx-ssh-install] step 4: stage dispatcher template + substitute on remote
[tmx-ssh-install] step 5: install authorized_keys entry (idempotent by fin
[remote] appended authorized_keys entry
[tmx-ssh-install] step 6: install Host alias 'nezha.local-tmx' in local ~/
[tmx-ssh-install] step 7: verification probe (token deliberately fails dis
[tmx-ssh-install] verification PASS: dispatcher rejected the probe token a
[tmx-ssh-install] step 8: summary
```

4.2 Custom remote project path

If you cloned the repo somewhere other than `~/Projects/tmux` on the target:

```
bash scripts/tmx-ssh-install.sh milosvasic@nezha.local \
    --remote-project-path /opt/tmux
```

4.3 Dry-run preview (changes nothing)

```
bash scripts/tmx-ssh-install.sh --dry-run milosvasic@nezha.local
```

Every mutating call (`ssh-keygen`, `scp`, `ssh "$AK_SCRIPT"`, `>> ~/.ssh/config`) prints `DRY-RUN would …` to stderr instead of running.

4.4 Force-overwrite a clobbered key

```
bash scripts/tmx-ssh-install.sh --force milosvasic@nezha.local
```

Use only if you know `~/.ssh/id_tmx_nezha_local` is wrong; the installer defaults to **never** overwriting an existing key.

5. Verification

After install, the Host alias is `<host>-tmx`, where `<host>` is the exact hostname you passed. For `milosvasic@nezha.local`, that's `nezha.local-tmx`. If your `~/.ssh/config` has shorter `Host nezha` aliases for the same machine, the dispatch alias lives side-by-side and does **not** affect them.

5.1 Login shell (no session arg)

```
$ ssh nezha.local-tmx
Linux nezha 6.12.x #1 SMP ...
Last login: Mon May 22 14:00:00 2026
$ # ← you are in a plain interactive bash login.
$ exit
```

The dispatcher saw empty `$SSH_ORIGINAL_COMMAND` and exec'd `bash -l`. Your `~/.bashrc` runs (including `tmx-shell-init.sh`), so you'll see the usual `[tmx] Enter session name …` prompt.

5.2 Attach to or create a named session

```
$ ssh nezha.local-tmx work
# → you are now inside tmx session 'work' on nezha.
# If 'work' existed, you attached to it (cwd is whatever its pane
# currently has). If it didn't, it was created with -c <recalled-
        pwd>.
```

5.3 Invalid name (positive test that the regex bites)

```
$ ssh nezha.local-tmx "echo hi"
[tmx-ssh-dispatch] this key accepts only a tmx session name
        (1-64 chars,
        [A-Za-z0-9_.-]). Use your normal SSH key for shell
        commands. Got: 'echo hi'
$ echo $?
1
```

Exactly what test `36_dispatcher_rejects_multiword.sh` asserts.

6. Uninstall

6.1 Just remove the wiring (keep the key)

```
bash scripts/tmx-ssh-install.sh --uninstall
        milosvasic@nezha.local
```

This deletes the `authorized_keys` line on the remote (matched by key fingerprint) and removes the `Host nezha.local-tmx` block from your local `~/.ssh/config`. The key files at `~/.ssh/id_tmx_nezha_local{,.pub}` are kept in case you want to re-wire later.

6.2 Remove key files too

```
bash scripts/tmx-ssh-install.sh --uninstall
        milosvasic@nezha.local --purge-key
```

7. Security notes

- The dispatch key is **single-purpose** — `command=` in `authorized_keys` restricts the key to running the dispatcher only. An attacker with the private key cannot get a shell, run arbitrary commands, port-forward, X11-forward, or agent-forward (all three are explicitly denied in the `authorized_keys` options).
- The dispatcher itself accepts only `[A-Za-z0-9_.-]{1,64}` as `SSH_ORIGINAL_COMMAND`; everything else is rejected with stderr + exit 1. There is no `eval`, no command substitution, no expansion of `$session` in any context where a shell metachar could escape.
- The local key file is `0600`; `~/.ssh` is `0700`. The installer enforces these modes on every run (`chmod` is idempotent).
- §11.4.10 forbids tracking credentials in git. The key files live in `~/.ssh/`, NOT in the repo. The dispatch script template lives in the repo (it's a public POSIX shell snippet); the deployed copy on the remote is mode 0755.

8. Worked example — end-to-end against `milosvasic@nezha.local`

Assume:

- You're on macOS, project cloned at `/Users/milosvasic/Projects/tmux`.

- Nezha runs Linux, project cloned at `/home/milosvasic/Projects/tmux`, `bash scripts/setup.sh` GREEN.
- Your existing `ssh milosvasic@nezha.local` works (any auth path).

```
# 1. From your mac, run the installer.
$ cd /Users/milosvasic/Projects/tmux
$ bash scripts/tmx-ssh-install.sh milosvasic@nezha.local
... (output as in §4.1) ...
[tmx-ssh-install] verification PASS: dispatcher rejected the
        probe token as designed
[tmx-ssh-install] step 8: summary

[tmx-ssh-install] install complete.

  local key:        /Users/milosvasic/.ssh/id_tmx_nezha_local
        (+ .pub)
  local alias:      Host nezha.local-tmx in ~/.ssh/config
  remote dispatch:  /home/milosvasic/Projects/tmux/scripts/tmx-
        ssh-dispatch.sh
  remote authkey:   ~/.ssh/authorized_keys (one entry, marked
        tmx-dispatch-nezha_local)

Usage:
  ssh nezha.local-tmx           # empty command -> interactive
        login shell
  ssh nezha.local-tmx work      # attaches/creates tmx session
        'work' with restored cwd

Uninstall:
  /Users/milosvasic/Projects/tmux/scripts/tmx-ssh-install.sh --
        uninstall milosvasic@nezha.local [--purge-key]

# 2. Verify the no-arg path: plain login.
$ ssh nezha.local-tmx
[tmx] Enter session name (blank or "default" = bare shell):
        ← .bashrc ran on the far side
default
$ exit

# 3. Verify the session path: open / create / attach.
$ ssh nezha.local-tmx work
[milosvasic@nezha ~]$ pwd
/home/milosvasic            ← first time → recall returned empty
        → $HOME
[milosvasic@nezha ~]$ cd /tmp
[milosvasic@nezha /tmp]$ # Press Ctrl-b d to detach.
[detached (from session work)]
```

```
# 4. Detach fired the cwd-capture hook. Reconnect and observe
        restored cwd.
$ ssh nezha.local-tmx work
[milosvasic@nezha /tmp]$ pwd
/tmp                          ← restored from ~/.tmx/state.json
```

9. Troubleshooting

Symptom	Diagnosis	Fix
`cannot reach milosvasic@nezha.local non-interactively`	Step 3 reachability probe failed	Run `ssh-copy-id milosvasic@nezha.local`, retry; ensure ssh-agent has your normal key
`remote project dir not found at ~/Projects/tmux/scripts`	Repo not cloned on remote, or different path	Clone there OR pass `--remote-project-path /actual/path`
`verification FAIL: probe did not produce expected [tmx-ssh-dispatch] stderr`	Remote sshd refused the dispatch key OR the script wasn't installed mode 0755	Check remote `ls -la ~/.ssh/authorized_keys` (mode 600); `journalctl -u sshd | tail -50` for sshd's reason
`ssh nezha.local-tmx work` hangs	Network blocked, or sshd config restricts `command=` keys	`ssh -v nezha.local-tmx work 2>&1 | head -50`; also `journalctl -u sshd` on the remote
`ssh nezha.local-tmx work` runs but lands in `$HOME` not the recorded cwd	`tmx-state` not on remote PATH under non-interactive sshd	The dispatcher already falls back to `$PROJECT/scripts/tmx-state-bin`; check it exists + is +x on the remote
`ssh nezha.local-tmx work` reports `tmx wrapper not found`	`tmx` not on remote PATH AND `scripts/tmx` not present	Re-run `bash scripts/setup.sh` on the remote to regenerate `scripts/tmx`
Re-running the installer prints "already present; skipping" for every step	Idempotent — this is correct	No action; this is the design
`~/.ssh/config` ends up with two `Host nezha.local-tmx` blocks	A previous half-uninstall left a stray block, then a fresh install added more	Hand-edit `~/.ssh/config` to keep one block; the installer's grep-by-`^Host` is fingerprint-equivalent

9.1 Server-side debug

```
# On nezha — watch sshd while you connect from the client.
$ sudo journalctl -u sshd -f
```

```
# In a second terminal on the client:
$ ssh -v nezha.local-tmx work 2>&1 | tee /tmp/ssh-verbose.log
```

Common sshd lines:

- `Accepted publickey for milosvasic from <ip> port <port> ssh2: ED25519 SHA256:...` — dispatch key was accepted.
- `error: PAM: …` — your sshd is using PAM, and PAM denies non-interactive sessions; this is rare but happens on some Linux configs. Either fix PAM or use a non-PAM auth path.

9.2 Client-side debug

```
$ ssh -vv nezha.local-tmx work 2>&1 | grep -E 'debug2: pubkey|
        Authenticated|Sending command'
debug2: pubkey: /Users/milosvasic/.ssh/id_tmx_nezha_local
Authenticated to nezha.local ([198.51.100.42]:22) using
        "publickey".
Sending command: work
```

If `pubkey` shows the wrong file, your `~/.ssh/config` Host block was not loaded — check syntax with `ssh -G nezha.local-tmx | grep -i identityfile`.

10. Cross-references

- docs/guides/tmx-shell-integration.md — the rc-side prompt
- docs/guides/tmx-state.md — the Go state daemon
- docs/manual/tmx-shell-integration.md — end-user master manual
- docs/scripts/tmx-ssh-install.md — §11.4.18 companion (installer internals)
- docs/scripts/tmx-ssh-dispatch.md — §11.4.18 companion (dispatcher internals)
- Spec: `docs/superpowers/specs/2026-05-22-tmx-shell-session-resume-design.md` §4.C + §4.D + §5.2

11. Anti-bluff (§11.4)

- Test 31 `ssh_dispatch_local.sh` — drives the dispatcher with `SSH_ORIGINAL_COMMAND=work`; positive evidence: `tmux ls | grep work` shows the created session.
- Test 32 `ssh_dispatch_remote_nezha.sh` — real SSH against `nezha.local`; SKIPs per §11.4.3 if the host is unreachable; otherwise asserts session created AND cwd restored from a pre-positioned state-file entry.
- Test 34 `ssh_install_idempotent.sh` — runs the installer twice; `grep -c "Host nezha.local-tmx" ~/.ssh/config` and the remote `authorized_keys` dup-count MUST both be 1, not 2.
- Test 36 `dispatcher_rejects_multiword.sh` — `SSH_ORIGINAL_COMMAND="echo hi"` produces stderr + exit 1, no tmux process spawned.
- Layer-4 paired mutations M22 (strip `command=` from the authorized_keys template → test 31 FAILs) and M23 (strip the regex validation → test 35 FAILs).

12. Last verified

2026-05-22 against Darwin arm64 client + Linux ALT 11 remote (`milosvasic@nezha.local`, kernel 6.12, systemd 258, OpenSSH 9.6), both running tmx 1.0.9.